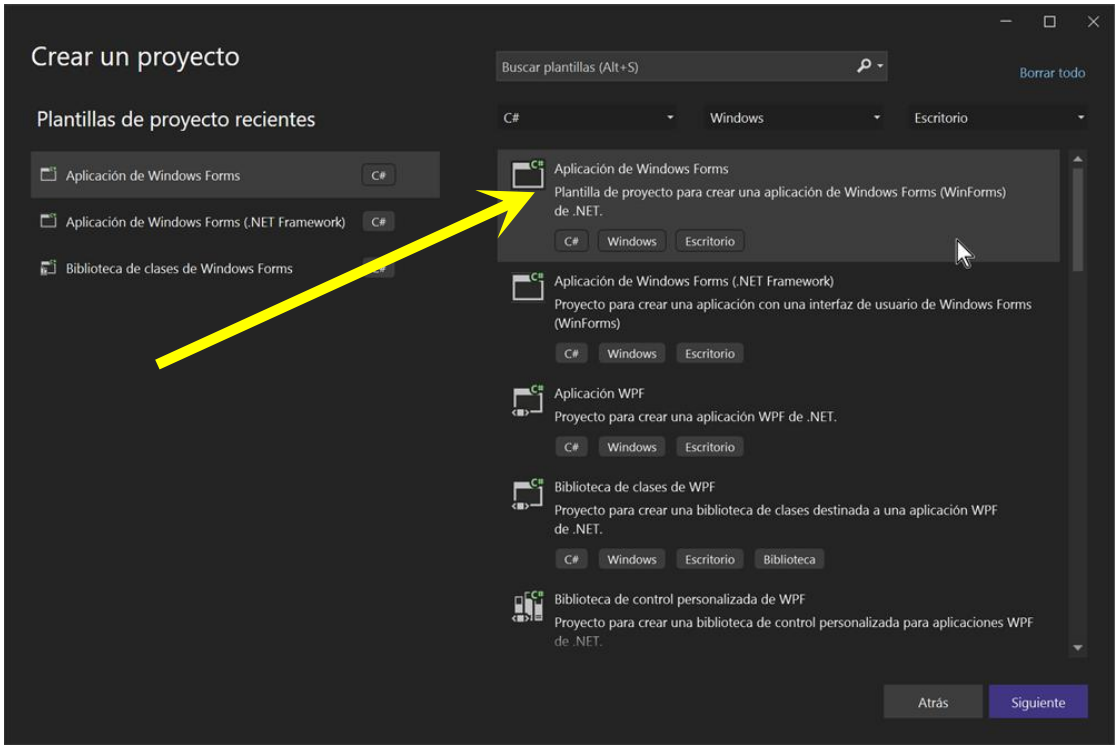
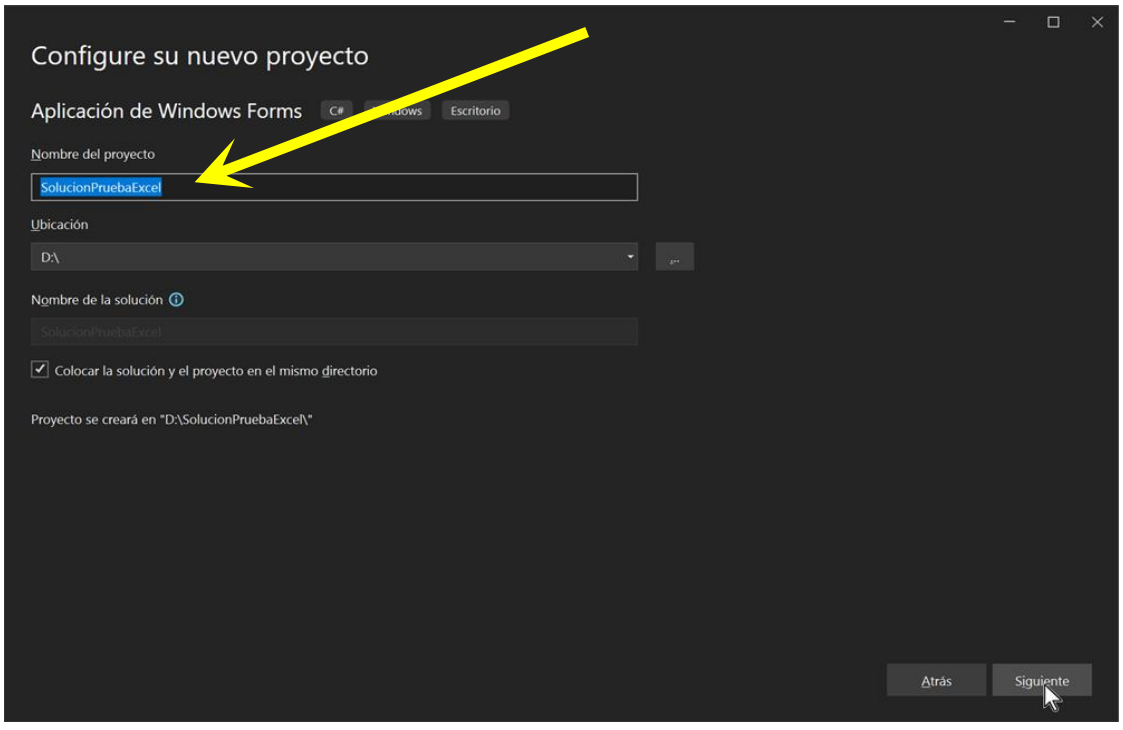
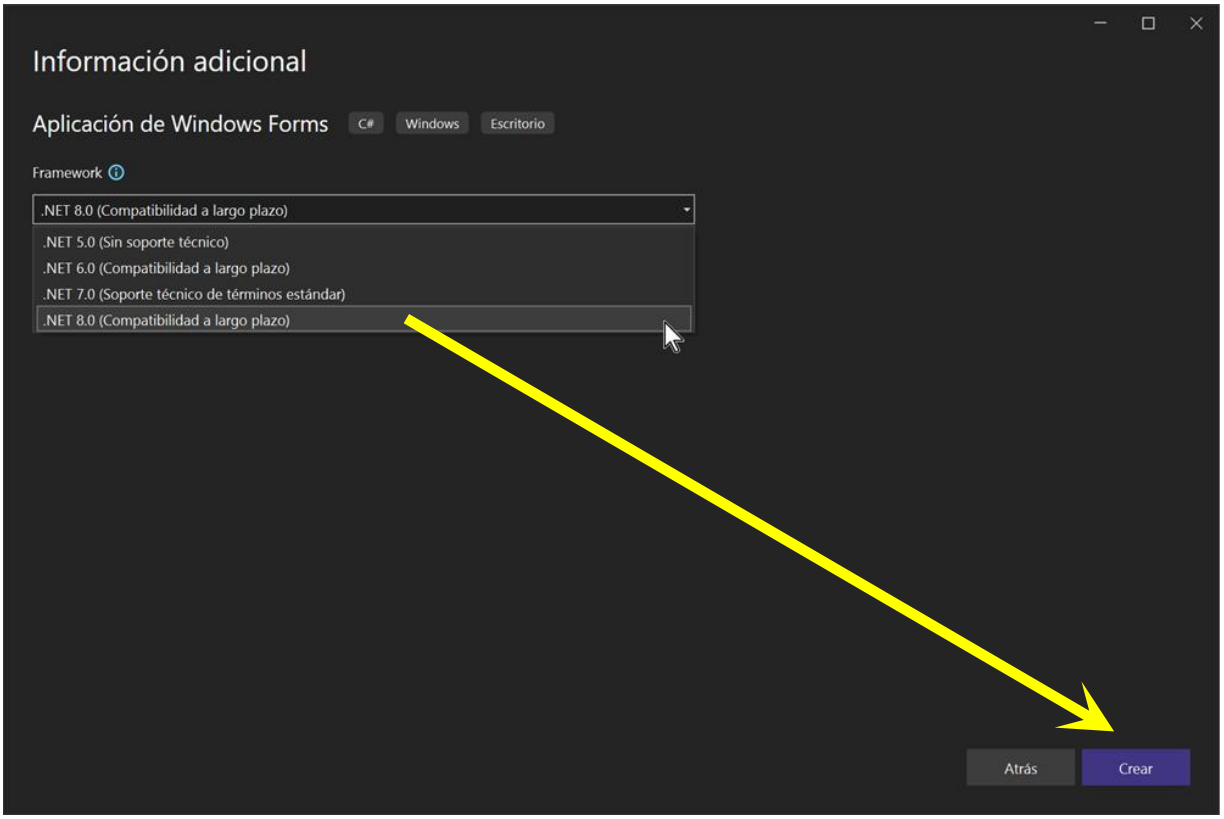
Learning to work with Excel

This document shows step by step:
1) Select exactly the first entry (NOT THE SECOND)



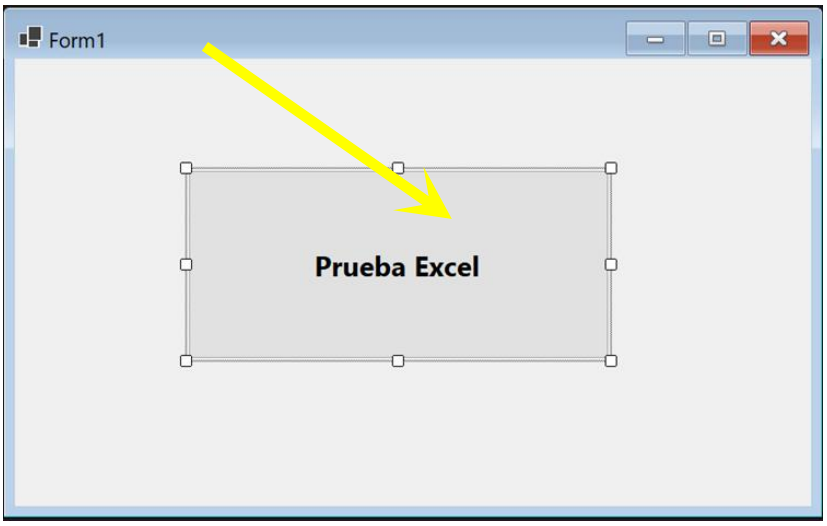
Now enter the name of the project TestExcel and click Next:



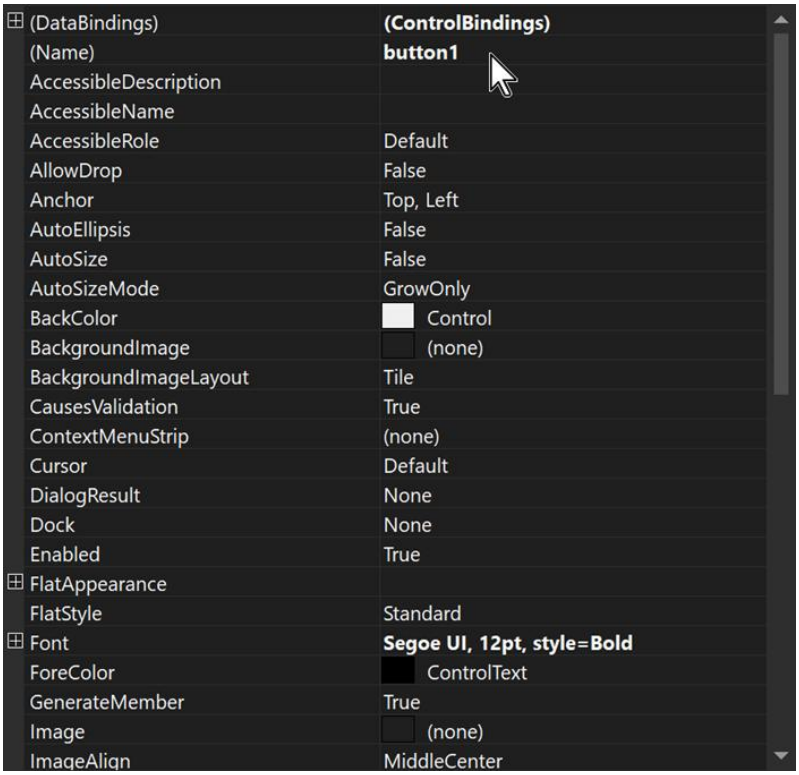


Now choose the Framework with which you are going to work, put .NET 8.0, which is the one Microsoft announces it as compatibility for future versions regardless of whether presents the latest updates:

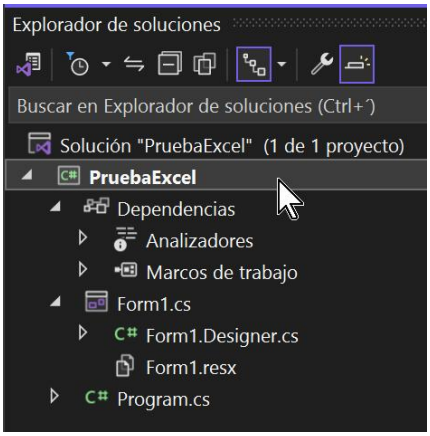
Immediately after the new project is created, assign a button (button1), with the name Try Excel and double click on it:



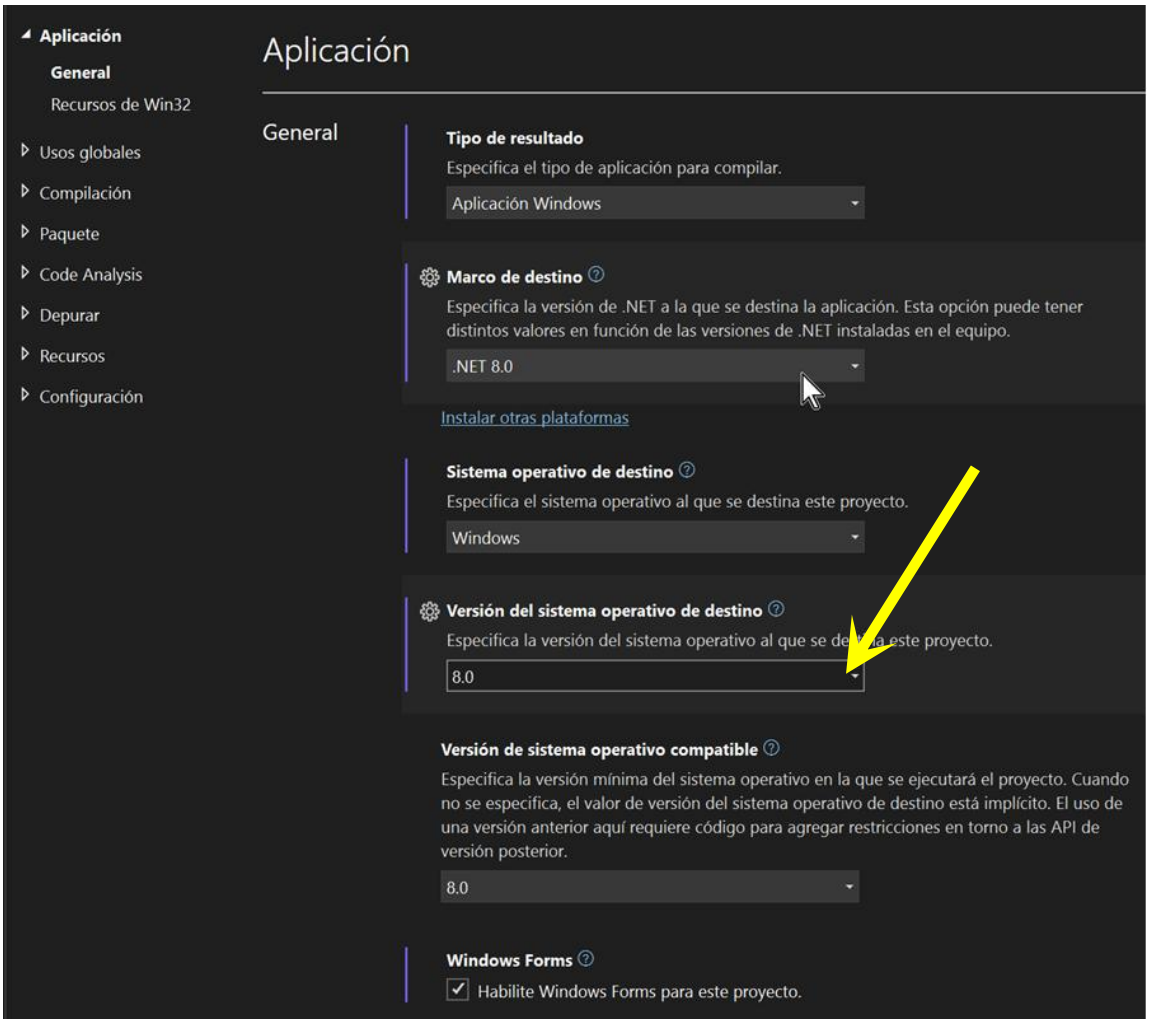
The properties window appears as follows:



You will see the Solution Explorer appear for the first time as follows:



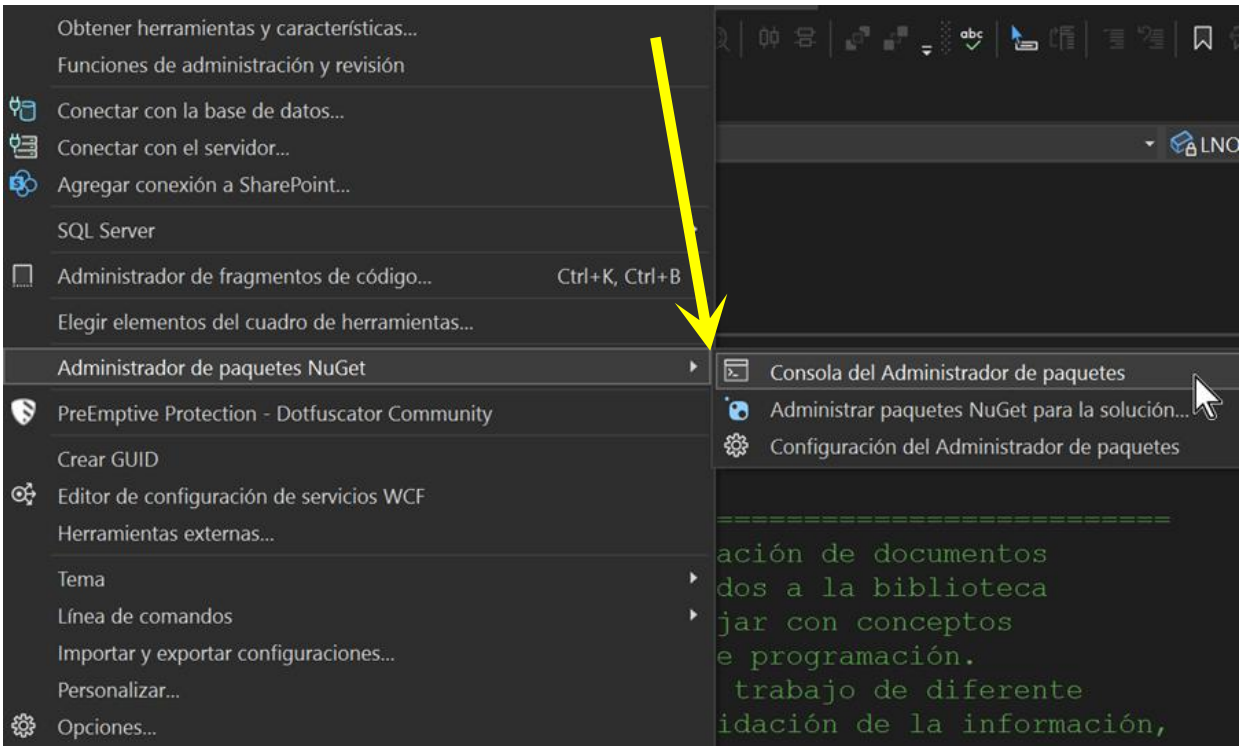
Now when you right click on the name of the project and request its properties, it appears in the center window, all the properties of said project as shown below:



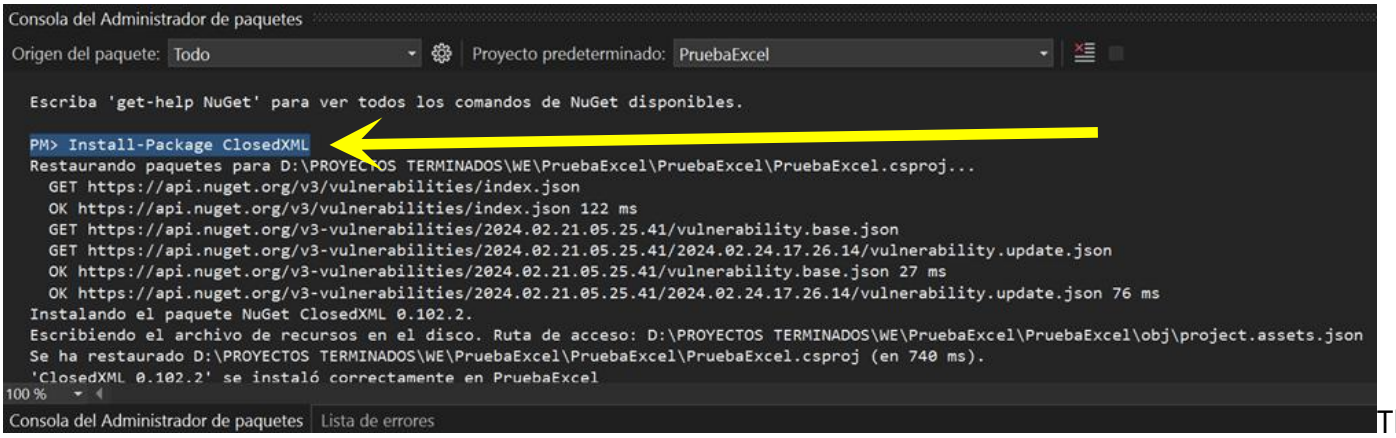
Select version 8.0 on all ComboBoxes in the image above. This will ensure that we are working with .NET 8.0 at all levels.

Well, now we start with the adaptation of the project so that it can work with Excel documents, for this reason we begin by adding the Nuget package (Package), following the following steps:

We click on Tools from the top menu of Visual Studio in order to select the Excel package that we are going to work with:

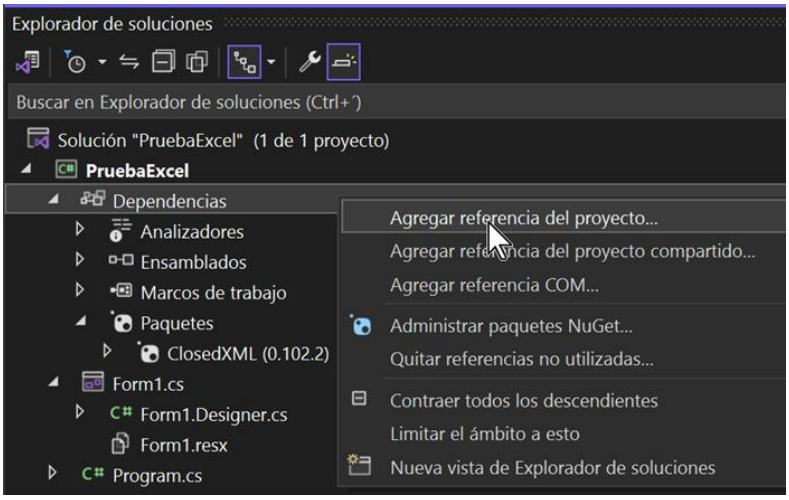


Next, add the package called ClosedXML and press Enter

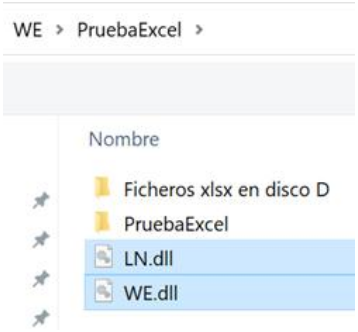


The messages associated with this package addition process appear below of Excel.

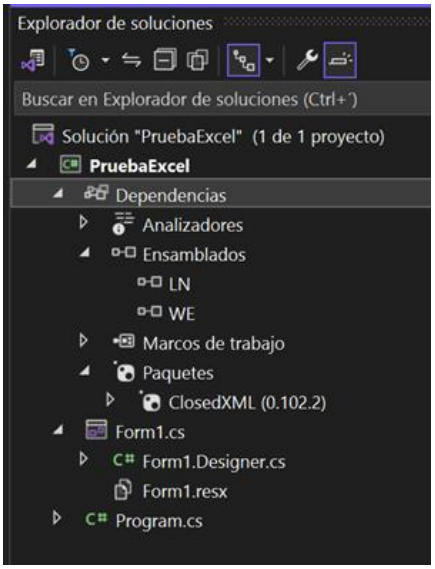
From this moment on, we proceed to add the two .DLLs that the program will use:



Click the Browse button and choose these two DLLs, and we add them to the project:



You should observe the new additions in the Solution Explorer, on the one hand the two LN (Library New) and WE (Word & Excel) libraries within the assemblies, and on the other the new CloseXML package that contains all the necessary components for Excel.



==========

Now completely replace all the content of the code page that now appears

originally this way:

```
namespace ExcelTest
{
public partial class Form1 : Form
	{
	public Form1()
		{
		InitializeComponent();
		}

	private void button1_Click(object sender, EventArgs e)
		{

		}
	}
}
```

==========

by the example program code that we attach below:

==========

```
using ClosedXML.Excel;
using DocumentFormat.OpenXml.Vml;
usingSystem;
using System.Windows.Forms;
using LN;
using WE;
namespace ExcelTest
{
//==========
= ==========
// Author: Lic. Hector Fortun (January 24, 2024) //==========
==========
// Example program for creating and modifying documents
// EXCEL under the use of methods added to libraries
// LN (Library New) created to work with concepts that are usually
// consume a lot of programming time, just like with the library
// WE (Word & Excel). The latter includes the methods designed to
// work with both Word and Excel and their non-Microsoft equivalents,
// like Ashampoo Office and others.
// LN on the other hand, covers several job profiles of different
// type such as, for example, information validation methods,
// methods for messaging, working with date and time, procedure calls
// and Windows objects, Error or Informational Messages, etc.
// All of the above, always with the main objective of making difficult things easy and saving time.
// The main objective has been to create a simple and at the same time sufficient logic to facilitate
// to the C# programmer under Visual Studio the programming work.
//


// Important note:
```

```
//
// Since originally being able to program for MSExcel required
// that MSOffice was installed on your PC by using a library
// required by Microsoft (Interop), since we decided to rely on
// some Nuget Packages, such as DocumentFormat.OpenXml (for Word) and
// ClosedXML (for EXCEL), both under the MIT license.
// Given my personal experience with the previous ones, I observed that
// they were a little reluctant to break the inertia and I decided to structure these
// tools in a very simple way that will help the programmer to
// advance quickly in the rapid completion of the most important tasks
// everyday.
// However, leaving the doors open so that when the
// programmer decides so use the original tools of the
// manufacturer, to whom we are very grateful for having the idea
// to be able to work with equivalents to Microsoft Word and Excel
// without having to have them installed on our PC.
//
// It is convenient to clarify that both packages allow free use, also
// in commercial applications.
//==========================================================================
= ===============
public partial class Form1 : Form
{
//------------------------------------------- -------
// Global Variables Zone
//------------------------------------------- -------
LN.LN       LNOBJ       = new LN.LN();
WE.WE       OBJ         = new WE.WE();
//------------------------------------------- -------
int C = 0;
int F = 0;
string B2 = "";
string B3 = "";
string B4 = "";
string cell = "";
float B2Float = 0;
float B3Float = 0;
float B4Float = 0;
float B6Float = 0;
string DataType = "";
XLDataType XX = new XLDataType();
//------------------------------------------- ------------
public Form1()
{
InitializeComponent();
```

```
}

private void button1_Click(object sender, System.EventArgs e)
{
try
{
OBJ.ExcelCreateDocExcel(@"D:\YYYNew.xlsx", "Sheet1");
// Open the file created in the previous step to
// work with he.
OBJ.ExcelFile = OBJ.ExcelOpenFile(@"D:\YYYNew.xlsx");
// Open the spreadsheet (sheet) and assign it to
// ExcelSheet
OBJ.ExcelSheet = OBJ.ExcelOpenSheet(1);

OBJ.ExcelFormatTitles(1, "NAME", "Segoe UI", 18, true, true,
XLColor.Red, XLColor.Yellow);
OBJ.ExcelFormatTitles(2, "SALARY", "Segoe UI", 18, true, true,
XLColor.Red, XLColor.Yellow);
OBJ.ExcelFormatCell(2, 1, "Arial", "Arial", 18, true, false,
XLColor.DarkBlue, XLColor.Turquoise);
OBJ.ExcelFormatCell(3, 1, "Times New Roman", "Times New Roman",
16, true, false,
XLColor.Black, XLColor.White);
OBJ.ExcelFormatCell(4, 1, "Courrier New", "Courrier New",
14, true, false,
XLColor.Red, XLColor.Yellow);
OBJ.ExcelFormatCell(2, 2, "250.50", "Times New Roman",
18, true, false,
XLColor.Turquoise, XLColor.DarkBlue);
OBJ.ExcelFormatCell(3, 2, "300.48", "Arial", 16, true, false,
XLColor.White, XLColor.Black);
OBJ.ExcelFormatCell(4, 2, "600.44", "Segoe UI", 14, true, false,
XLColor.Yellow, XLColor.Red);

OBJ.ExcelSheet.Row(1).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(2).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(3).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();
```

```
OBJ.ExcelSheet.Row(4).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(5).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

OBJ.ExcelSheet.Row(6).AdjustToContents();
OBJ.ExcelSheet.Column(1).AdjustToContents();
OBJ.ExcelSheet.Column(2).AdjustToContents();

//=======ARITHMETIC OPERATIONS WITH CELLS IN A COLUMN=======
B2 = OBJ.ExcelSheet.Cell("B2").Value.ToString();
B3 = OBJ.ExcelSheet.Cell("B3").Value.ToString();
B4 = OBJ.ExcelSheet.Cell("B4").Value.ToString();
//------------------------------------------------ ------------------
B2Float = float.Parse(B2);
B3Float = float.Parse(B3);
B4Float = float.Parse(B4);
//------------------------------------------------ ------------------
B6Float = B2Float + B3Float + B4Float;
//------------------------------------------------ ------------------
OBJ.ExcelSheet.Cell("B6").Value = B6Float.ToString();

//------------FORMAT THE RESULT COLUMN---------
OBJ.ExcelFormatCell(6, 2, OBJ.ExcelSheet.Cell("B6").Value.ToString(),
"Arial", 18, true, false,
XLColor.DarkRed, XLColor.Yellow);
//------------------------------------------------ ------------------
// The result of this arithmetic operation "(B4 + B6 + A7) / 3"
// is in A8
//------------------------------------------------ ------------------
// It is convenient to note that cell B5 has no content,
// however it is accepted in the operation as if the value were zero.
//------------------------------------------------ ------------------
OBJ.ExcelFormula("A8", "(B4 + B5 + B6 + A7) / 3");



//======ASSIGNING A DATA TYPE TO A CELL=====
OBJ.ExcelSheet.Cell(7,1).Value = "123.45";
B2 = OBJ.ExcelSheet.Cell(7,1).DataType.ToString();
XX = OBJ.ExcelSheet.Cell(7,1).DataType;
```

```
OBJ.ExcelFormatCell(7, 1, OBJ.ExcelSheet.Cell(7,1).Value.ToString(),
"Arial", 14, true, false,
XLColor.DarkRed, XLColor.Yellow);
cell = OBJ.ExcelGetCell("A7");
XX = XLDataType.Number;

if (XX == XLDataType.Text)
{
OBJ.MensINFO("Cell A7 = Text");
}
if (XX == XLDataType.Number)
{
OBJ.MensINFO("Cell A7 = Number");
}
//---------------------------------------------- ------------------
OBJ.ExcelWriteCellRC(5, 1, "AAA");
OBJ.ExcelWriteCellRC(6, 1, "BBB");

OBJ.ExcelWriteCell("B3", "CCC");
OBJ.ExcelWriteCell("B2", "DDD");
// Get the cell value (3,2)
cell = OBJ.ExcelGetCellRC(3, 2);
//Get the Data type of (4,2)
DataType = OBJ.ExcelGetCellDataTypeRC(4, 2);
//Get the Data type of "B2"
cell = OBJ.ExcelGetCellDataType("B2");

OBJ.MensINFO("# Columns: " + C.ToString() + "\n" +
"# Rows: " + F.ToString() +
"\nCell(3,2) = " + cell +
"\nDataType (4,2) = " + DataType +
"\nData Type (2,2) = " + cell);

OBJ.MensINFO("End of Process");
}
catch
{
OBJ.MensINFO("Exception Detected - The program is cancelled\n" +
e.ToString());
Application.Exit();
}
OBJ.ExcelSaveFile(@"D:\AAAOutput.xlsx");
LNOBJ.MensINFO("Excel Test Completed Successfully" +
@"\nResulting File in D:\AAAOutput.xlsx");
Application.Exit();
```

```
}

}
}
```

And the configuration process to run the Excel example program is now complete.
Compile and run it and you will see after execution that in the root of the disk D:\
These two new files GENERATED BY THE PROGRAM appear:

